

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI Todo Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 3
 - 2.1 CLI Todo Manager Traceability Map 7
 - 2.2 CLI Command Interaction Sequence 8
 - 2.3 CLI Execution Workflow 9
 - 2.4 Todo Data Model 10
 - 3. Detailed Technical Specifications & Business Rules 11
 - 3.1 Requirements Traceability 12
 - 3.2 Security Rules 13
 - 3.3 Data Models 13
 - 4. Project Governance & Structural Gaps 13
 - 4.1 Structural Gaps 13
 - 4.2 Remediation & Workflow 14
 - 5. Technical & Domain Glossary (Terminology Reference) 14

CLI Todo Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI Todo Manager is a Python-based command-line utility designed for task lifecycle management. It utilizes a local JSON file (`~/ . todos . json`) as its primary data store to persist task IDs, descriptions, and completion statuses. The system provides core CRUD operations and bulk cleanup capabilities via a modular CLI interface.

1.2 Maturity Assessment

The project is **READY** for execution. While there are minor structural gaps regarding the absence of a formal checklist and explicit performance targets for large datasets, the functional specifications and test scenarios are comprehensive and logically mapped, ensuring a stable baseline for implementation.

1.3 Technical Stack

- Python 3.8+

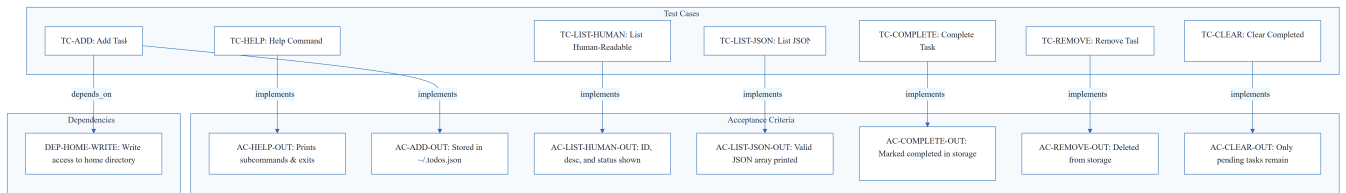
1.4 Architectural Constraints

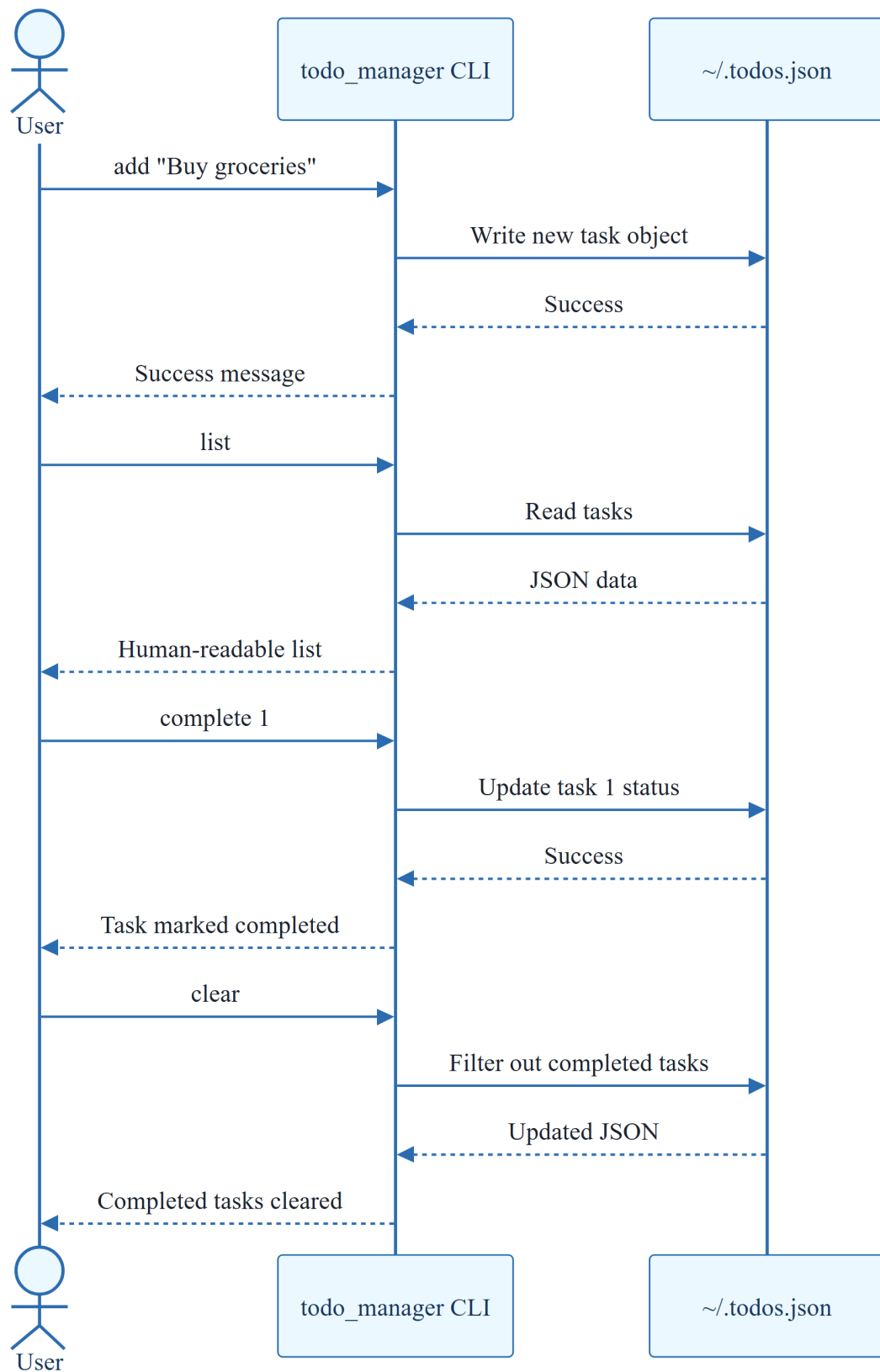
- **Storage:** Local persistence via `~/ . todos . json`.
- **Data Integrity:** Stop execution with user-friendly error on storage corruption; prohibit automatic overwriting of corrupted files.
- **Output Formats:** Support for both human-readable and valid JSON array formats.
- **File System:** Mandatory write access to the user's home directory.

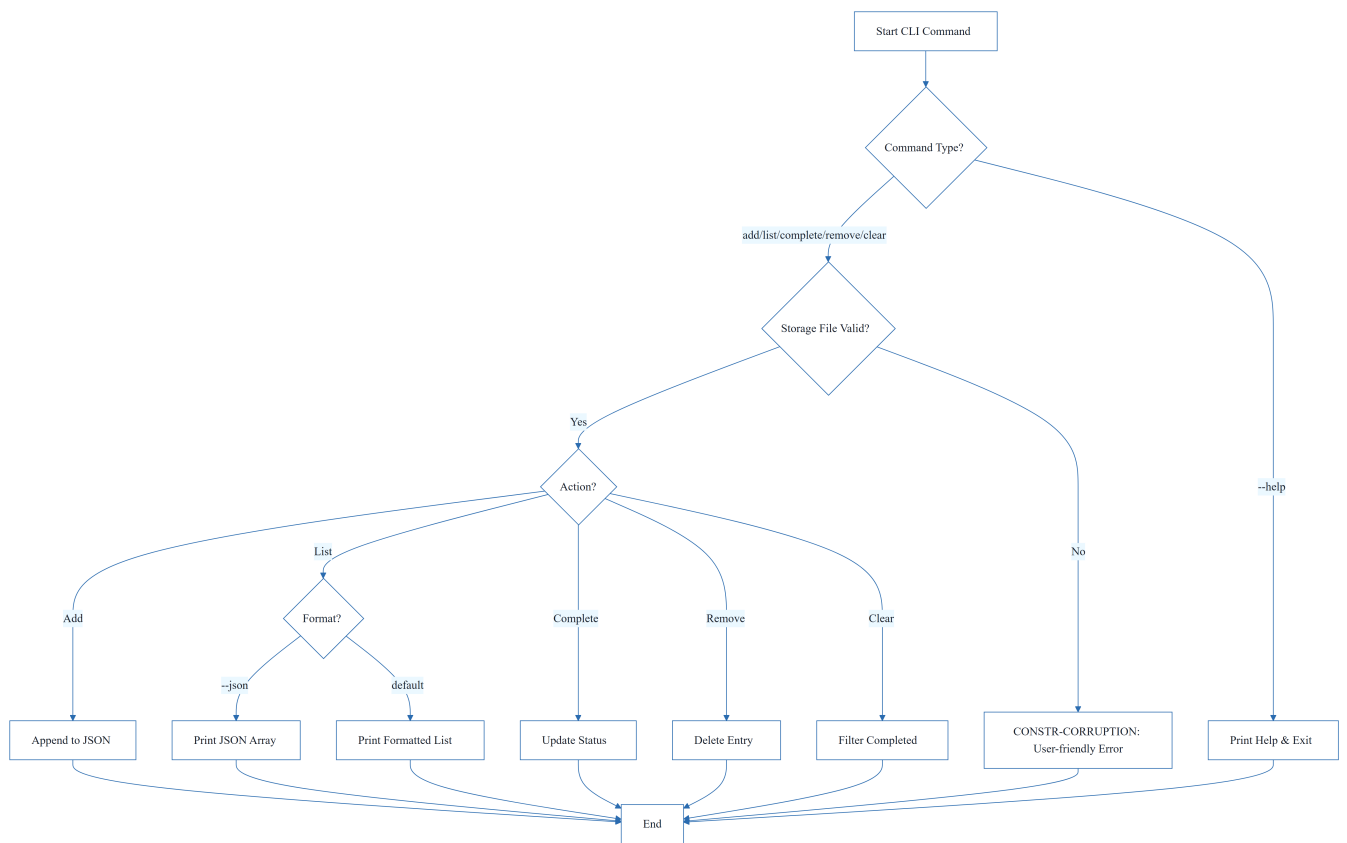
1.5 Critical Dependencies

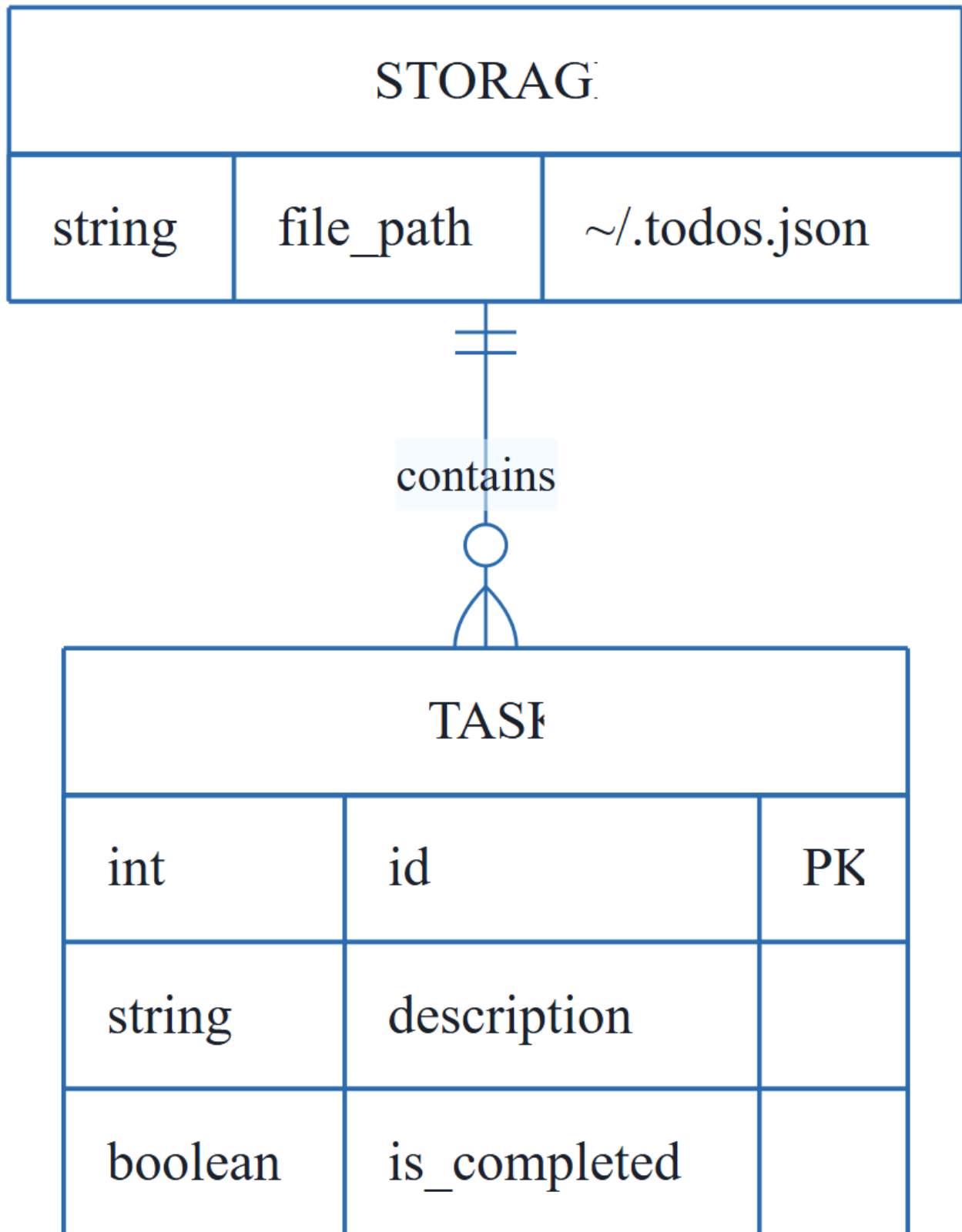
- Python 3.8 runtime.
- Home directory write permissions for `~/ . todos . json`.
- Referential integrity between CLI commands and `data-model.md` task shapes.
- Command behavior alignment with `contracts/commands.md`.

2. Architecture Workflows





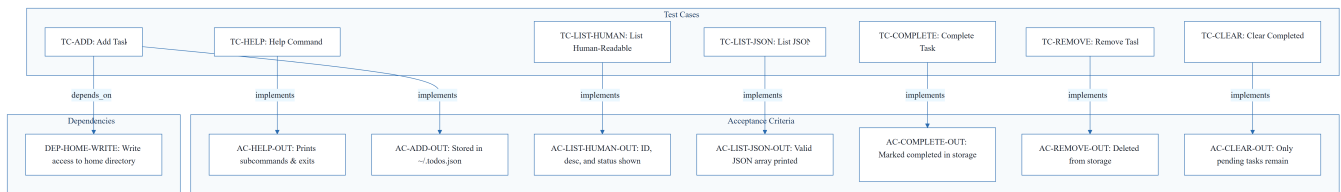




& Visual Diagrams

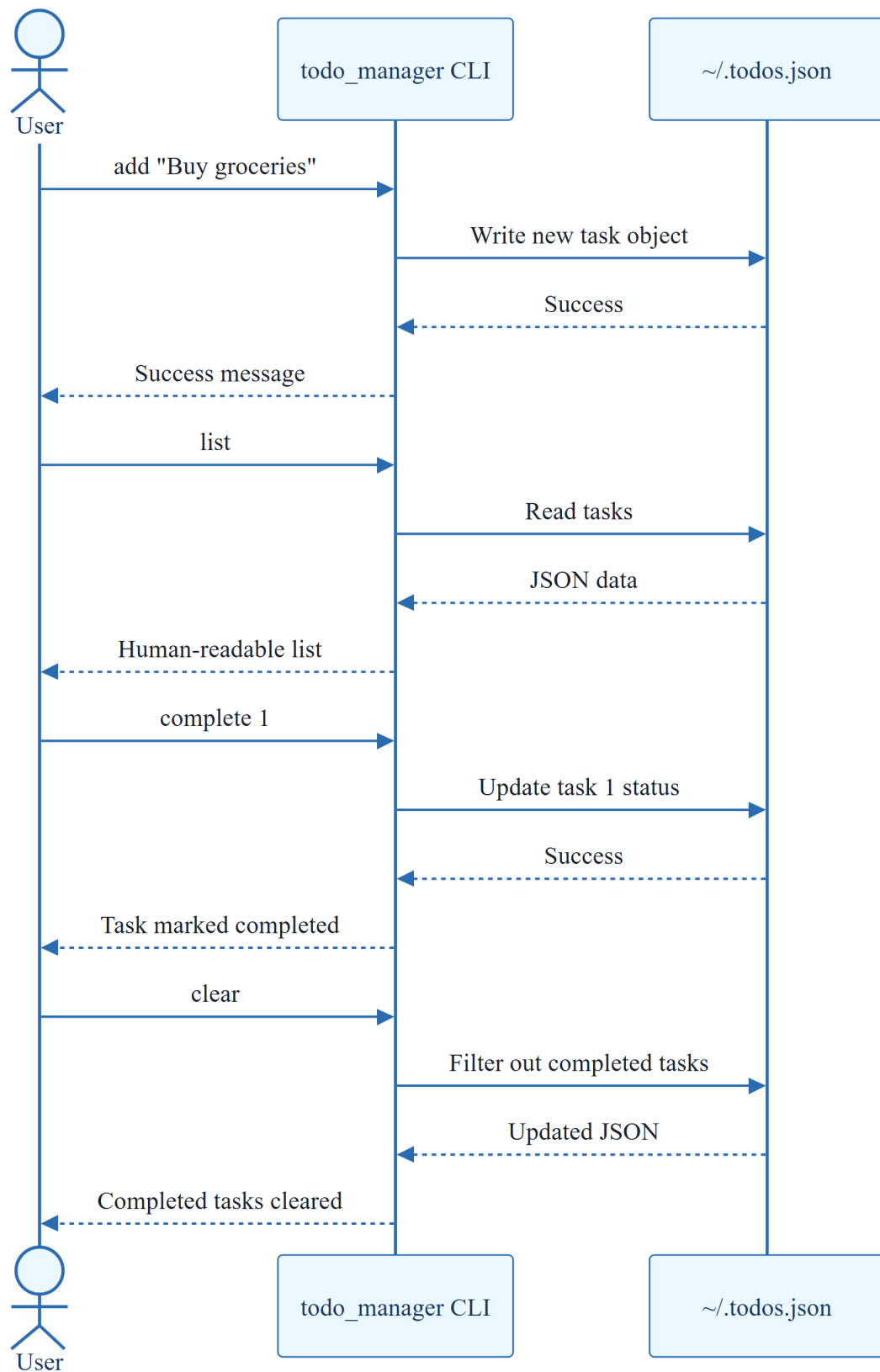
2.1 CLI Todo Manager Traceability Map

Maps test cases to their respective acceptance criteria and system dependencies for full traceability.



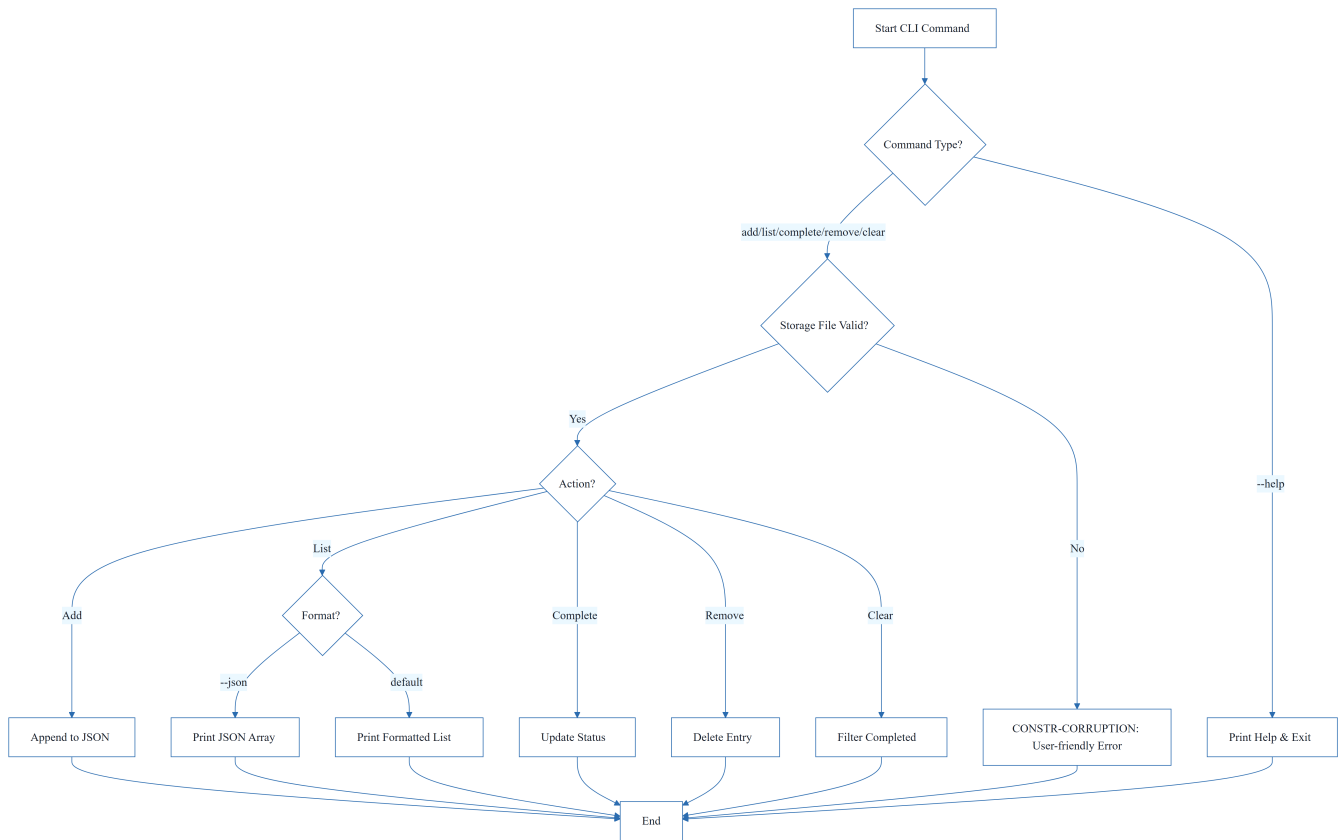
2.2 CLI Command Interaction Sequence

Models the interaction between the User, the CLI Application, and the JSON storage file for core operations.



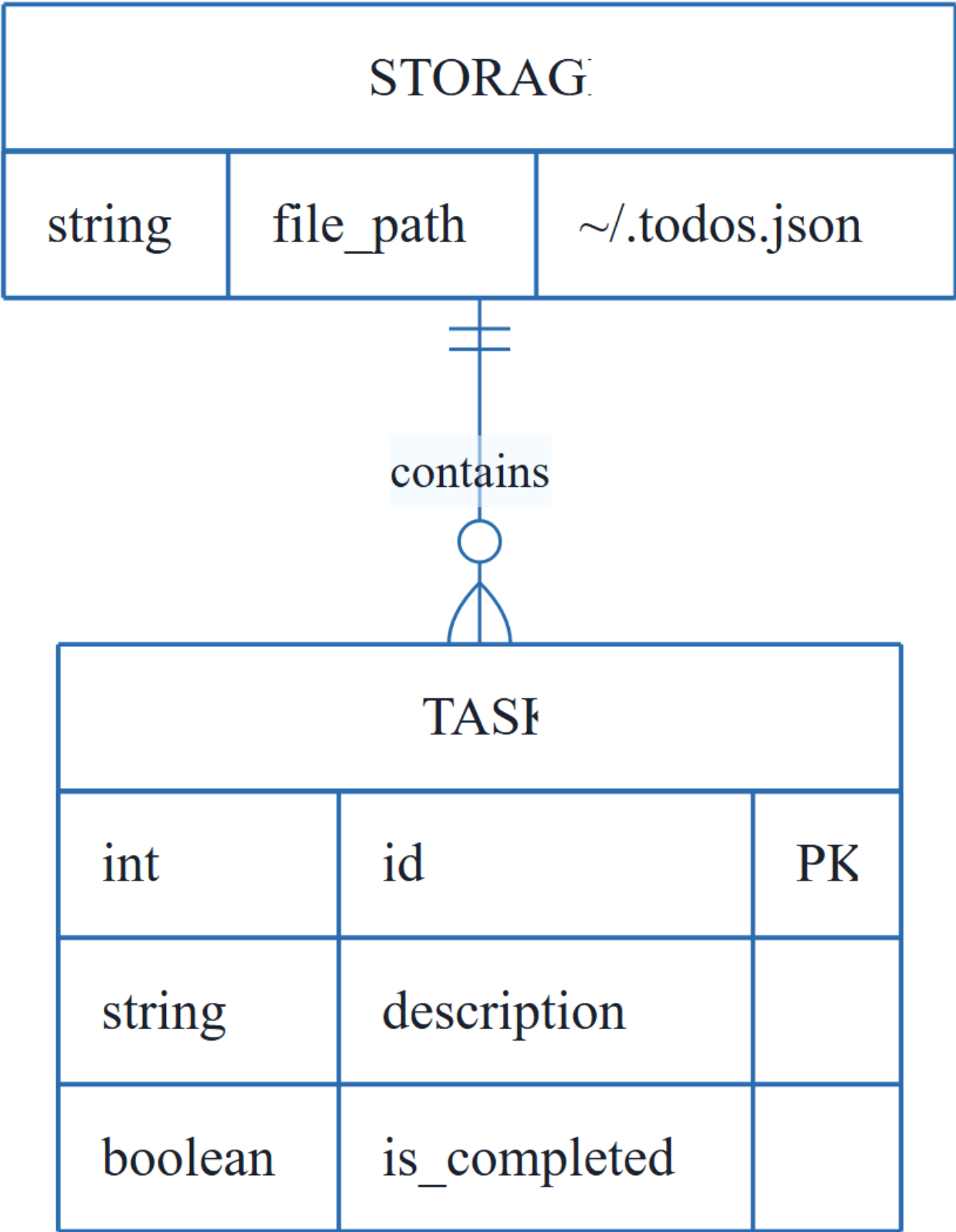
2.3 CLI Execution Workflow

Represents the internal logic flow of the CLI, including error handling for file corruption.



2.4 Todo Data Model

Represents the structure of the task storage as implied by the validation guide and acceptance criteria.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

Identifier	Type	Description	Acceptance Criterion / Dependency
DEP-PY38	dependency	Python 3.8 or newer	N/A
DEP-HOME-WRITE	dependency	Write access to home directory	N/A
TC-HELP	test_case	Confirm command help is available: <code>python -m todo_manager --help</code>	AC-HELP-OUT
AC-HELP-OUT	acceptance_criterion	CLI prints available subcommands and exits successfully	N/A
TC-ADD	test_case	Add a task: <code>python -m todo_manager add "Buy groceries"</code>	AC-ADD-OUT
AC-ADD-OUT	acceptance_criterion	Task stored in <code>~/ .todos.json</code> and success message shown	N/A
TC-LIST-HUMAN	test_case	List tasks in human-readable mode: <code>python -m todo_manager list</code>	AC-LIST-HUMAN-OUT
AC-LIST-HUMAN-OUT	acceptance_criterion	Task appears with ID, description, and status	N/A
TC-LIST-JSON	test_case	List tasks as JSON: <code>python -m todo_manager list --json</code>	AC-LIST-JSON-OUT
AC-LIST-JSON-OUT	acceptance_criterion	CLI prints valid JSON containing the tasks array	N/A
TC-COMPLETE	test_case	Complete a task: <code>python -m todo_manager complete 1</code>	AC-COMPLETE-OUT
AC-COMPLETE-OUT	acceptance_criterion	Task marked completed and updated in listings	N/A
TC-REMOVE	test_case	Remove a task: <code>python -m todo_manager remove 1</code>	AC-REMOVE-OUT
AC-REMOVE-OUT	acceptance_criterion	Task is deleted from storage	N/A
TC-CLEAR	test_case	Clear completed tasks: <code>python -m todo_manager clear</code>	AC-CLEAR-OUT

Identifier	Type	Description	Acceptance Criterion / Dependency
AC-CLEAR-OUT	acceptance_criterion	All completed tasks removed, pending tasks remain	N/A
CONSTR-CORRUPTION	constraint	If storage file is corrupted, CLI should stop with user-friendly error instead of overwriting	N/A

3.2 Security Rules

- **File System Access:** The application must strictly operate within the user's home directory for storage (~/.todos.json).
- **Error Handling:** In the event of file corruption, the system must fail-safe by stopping execution and notifying the user, preventing data loss via accidental overwrites.

3.3 Data Models

- **Storage Format:** JSON array of objects.
- **Task Object:**
 - `id` (Integer): Unique identifier.
 - `description` (String): Task text.
 - `is_completed` (Boolean): Completion status.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	LOW	The document contains 'Expected outcomes' within the test cases, but a dedicated high-level Acceptance Criteria section is missing.
Checkboxes Checklist	MEDIUM	Convert the 'Validate the CLI' steps into a formal checklist for execution tracking.
Security & Performance Constraints	LOW	Define performance targets for large todo lists or security constraints for the JSON file.
Open Questions & Uncertainties	LOW	No open questions were identified in the source.

4.2 Remediation & Workflow

The identified gaps are primarily documentation-related and do not impede the current implementation phase. It is recommended to implement the "Checkboxes Checklist" during the QA phase to ensure 100% test coverage of the defined TC- identifiers.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
ID	BUSINESS_DOMAIN	AC-LIST-HUMAN-OUT	A unique numeric reference assigned to each entry to enable targeted mutation or deletion.
JSON	TECHNICAL_STACK	AC-LIST-JSON-OUT	The structured data interchange format used for both persistent storage in the home directory and machine-readable output.
Python 3.8	TECHNICAL_STACK	DEP-PY38	The minimum required runtime environment version for executing the command line interface.